INDUSTRIAL AI-POWERED WORK ORDER & JOB CARD MANAGEMENT SYSTEM

Functional Requirements • Non-Functional Requirements • Testing • Deployment • Database • Viva Preparation

Live application:
https://ai-digital-work-order-management-system-1.onrender.com

Backend API:
https://ai-digital-work-order-management-system.onrender.com

Project guide prepared from the project details and testing steps discussed. It distinguishes verified activities from tests that still need to be performed.

1. Project Overview

The Industrial AI-Powered Digital Work Order & Job Card Management System is a full-stack web application intended to centralize industrial maintenance workflows. It supports work orders, job cards, equipment records, inventory, material requests, audit logs, and AI-assisted operational features.

Layer	Technology / Responsibility
Frontend	React, TypeScript, Vite, Tailwind CSS, React Router; user interface and navigation.
Backend	Node.js, Express.js, TypeScript; REST API and application business logic.
Database	PostgreSQL hosted on Render; persistent application data.
ORM	Prisma; schema definition, database queries, and migrations.
Authentication	JWT and bcrypt, with role-based access controls configured in the application.
Real-time	Socket.io integration for supported events.
Deployment	Render frontend static site, backend web service, and managed PostgreSQL database.

Live deployment links

- Frontend: https://ai-digital-work-order-management-system-1.onrender.com
- Backend service: https://ai-digital-work-order-management-system.onrender.com
- Database: hosted PostgreSQL instance on Render; it is not intended to be a public website.

2. Functional Requirements

Functional requirements describe what the system should do. The table below reflects documented project functionality; individual features should be tested before claiming that each one passes.

ID	Requirement	Description	Suggested verification
FR-01	Authentication	Users sign in using email and password.	Test valid and invalid credentials.
FR-02	Role-based access	Features are restricted according to the user's role.	Compare permissions across the four roles.
FR-03	User management	Administrators manage user accounts.	Open Users as Administrator; test allowed actions.
FR-04	Equipment	Equipment records and criticality are maintained.	Inspect equipment list and available actions.
FR-05	Work orders	Supervisors create and track maintenance work orders.	Create a valid work order and verify it appears.
FR-06	Input validation	Invalid work-order fields are rejected.	Submit a task description shorter than five characters.
FR-07	Job cards	Maintenance tasks and completion statuses are tracked.	Inspect job cards and update permitted fields.
FR-08	Material requests	Workers request materials for maintenance.	Submit a request using a Worker account, if enabled.
FR-09	Inventory	Inventory items and stock are managed.	Inspect inventory and test permitted stock actions.

ID	Requirement	Description	Suggested verification
FR-10	Inventory issuance	Issued items are recorded and stock is updated.	Issue material and verify stock changes, if implemented.
FR-11	Audit logs	Relevant activities are recorded for traceability.	Perform an auditable action and inspect Audit Logs.
FR-12	AI priority scoring	Work orders receive a score based on defined factors.	Vary deadline, criticality, and SLA-risk terms.
FR-13	Semantic search	Natural-language queries retrieve relevant work orders.	Search terms such as 'urgent turbine leak'.
FR-14	AI reporting	Operational metrics are summarized in natural language.	Compare report figures with database records.
FR-15	Real-time events	Socket.io communicates supported events.	Observe a supported event from a connected client.
FR-16	Logout	Users can end their session and return to login.	Log out and confirm the login page loads.

3. Non-Functional Requirements

Non-functional requirements describe how well the system should operate. Targets should be measurable where possible; do not claim a performance or security target was met without test evidence.

ID	Requirement	Application	Evaluation method
NFR-01	Performance	Pages and API requests respond in an acceptable time.	Measure response times using DevTools or an API client.
NFR-02	Security	Passwords and protected operations are handled securely.	Test invalid login, unauthorized API calls, and role restrictions.
NFR-03	Reliability	Errors are handled without crashing the application.	Submit invalid data and simulate API failures.
NFR-04	Data integrity	Records and relationships remain consistent.	Create records and verify persistence and relationships.
NFR-05	Usability	Users can complete common tasks with clear navigation.	Walk through workflows using each role.
NFR-06	Maintainability	Code is organized into frontend, backend, and data layers.	Inspect modules, TypeScript types, and project structure.
NFR-07	Scalability	The system can handle growth within infrastructure limits.	Conduct load tests and monitor resource usage.
NFR-08	Compatibility	The app works in target modern browsers.	Test supported browsers and viewport sizes.
NFR-09	Availability	Services are reachable when hosting and dependencies are available.	Check frontend, backend, and database connectivity.
NFR-10	Responsiveness	The interface adapts to desktop, tablet, and mobile.	Test multiple viewport widths.
NFR-11	Session handling	Invalid or expired sessions cannot access protected pages.	Invalidate a token and request a protected route.
NFR-12	Auditability	Important operations have traceable activity records.	Compare performed actions with audit records.

4. Environments Used for Testing

A. Local development

Tools: Visual Studio Code, Node.js, npm, TypeScript, Prisma. Local development is used to change code, run the frontend/backend, inspect logs, and debug issues before deployment.

Backend commands (from the backend directory):

```
npm install
npm run dev
```

Frontend commands (in a separate terminal):

```
cd frontend
npm install
npm run dev
```

Use the local URL printed by Vite; common default is http://localhost:5173. The configured port may differ.

B. Deployed frontend

URL: https://ai-digital-work-order-management-system-1.onrender.com

The live application was used to test login, logout, route handling, and work-order validation. After logout initially showed Not Found, a Render rewrite rule was configured to serve /index.html for application routes. The user confirmed that this issue was resolved.

C. Deployed backend

URL: https://ai-digital-work-order-management-system.onrender.com

The browser Network panel can be used to inspect requests such as POST /api/work-orders, including status codes, payloads, and response bodies. This helps isolate frontend, API validation, authentication, and database errors.

D. PostgreSQL database

The Render PostgreSQL database is accessed through Prisma. The schema was validated and the migration status reported that the database schema was up to date. The seed script ran successfully and Prisma Studio was used to inspect application records.

5. Testing Evidence and Observations

Activity	Observed result	What it supports
Prisma schema validation	npx prisma validate --schema=prisma/schema.prisma succeeded.	Schema syntax and model definitions validated.
Migration status	One migration found; database schema reported up to date.	Migration state checked against the configured database.
Database seeding	Seed script completed successfully.	Seed script could connect and insert sample data.
Prisma Studio	Application tables and records were visible.	Database contents could be inspected.
Logout route debugging	After configuring Render rewrite, user confirmed it worked.	Route fallback issue was resolved and retested.
Work-order validation	A task description under five characters produced a validation error.	Backend/API validation was observed.

Evidence limitation: These observations do not prove every requirement passes. Authorization across all roles, automated test suite results, performance, load, and security testing require separate verification.

6. Testing Types

Testing type	Purpose	How it applies
Functional	Verify individual features.	Login, work orders, job cards, inventory, reports.
Integration	Verify components work together.	React frontend → Express API → Prisma → PostgreSQL.
Validation	Reject malformed or incomplete input.	Short work-order descriptions and missing fields.
Database	Verify persistence, schema, and relationships.	Prisma validation, migration status, Studio inspection.
Authentication	Verify identity and session handling.	Valid/invalid login, logout, invalid token.
Authorization	Verify role-specific permissions.	Attempt restricted pages and API operations.
Regression	Ensure fixes do not break prior behavior.	Retest login/logout and protected routes after changes.
Deployment	Verify hosted services work.	Open Render frontend and inspect API responses.
API	Verify endpoint requests and responses.	Browser DevTools or Postman.
Performance	Measure response times and load behavior.	Timed API checks and a controlled load test.
Security	Check for access control and common vulnerabilities.	Negative authorization tests and security scanning.

7. Tools and Their Uses

Tool	Purpose
Visual Studio Code	Code editing and local debugging.
Git / GitHub	Version control and source-code collaboration.
Render	Hosting frontend, backend, and managed database.
PostgreSQL	Persistent relational data storage.
Prisma ORM	Schema, queries, migrations, and client generation.
Prisma Studio	Visual inspection and management of records.
npm	Dependency installation and project scripts.
Chrome DevTools	Inspect browser errors, requests, and API responses.
TypeScript	Static type checking.
React / Vite	Frontend development and build tooling.
Node.js / Express	Backend runtime and API framework.
Socket.io	Real-time communication for supported events.
Jest	Automated tests, if configured tests are executed.

8. Database Inspection

From the backend directory, run:

```
npx prisma studio --schema=prisma/schema.prisma
```

Prisma Studio normally opens at http://localhost:5555. It uses the DATABASE_URL from the environment configuration. Confirm that it points to the intended database before opening Studio.

Model / Table	Purpose
User	Accounts and roles.
Equipment	Industrial assets and their details.
WorkOrder	Maintenance work orders.
JobCard	Maintenance task assignments.
MaterialRequest	Requests for materials.
InventoryItem	Stock and spare-parts records.
InventoryIssue	Issued inventory records.
AuditLog	Relevant activity records.

Caution: Prisma Studio can modify production records. Avoid destructive changes during a demonstration. Never display or share the database connection string or password.

9. Local Setup Guide

Prerequisites: Node.js/npm, Git, and a PostgreSQL database. A separate development database is recommended.

1. Clone the repository:

```
git clone https://github.com/Aaryansinngh/ai-digital-work-order-management-system.git
cd ai-digital-work-order-management-system
```

2. Install backend dependencies:

```
cd backend
npm install
```

3. Create backend/.env with the required variables. Example placeholders:

```
DATABASE_URL="YOUR_POSTGRESQL_CONNECTION_STRING"
JWT_SECRET="YOUR_LONG_RANDOM_SECRET"
PORT=5000
```

Check source code for any additional required variables. Never commit .env or production secrets.

4. Generate Prisma Client and check migration status:

```
npx prisma generate --schema=prisma/schema.prisma
npx prisma migrate status --schema=prisma/schema.prisma
```

5. Apply existing migrations only after confirming the target database:

```
npx prisma migrate deploy --schema=prisma/schema.prisma
```

6. Optional: seed a fresh development database after reviewing the seed script:

```
npm run seed
```

Do not assume seeding is safe to repeat. Do not run prisma migrate reset against production.

7. Start the backend:

```
npm run dev
```

8. In a second terminal, start the frontend:

```
cd frontend
npm install
npm run dev
```

Open the URL printed by Vite and ensure the frontend API configuration points to the intended backend.

10. Docker

If docker/docker-compose.yml exists and is correctly configured, from the repository root:

```
docker compose -f docker/docker-compose.yml up --build -d
```

Inspect the Compose file and environment settings before running it. The container setup may use a separate database and is not automatically connected to Render.

Stop containers with:

```
docker compose -f docker/docker-compose.yml down
```

11. Automated Testing

From backend, run:

```
npm test
```

The intended test coverage may include authentication, role-based authorization, work-order/job-card constraints, inventory stock safeguards, and AI scoring. Report only the tests and results actually shown by the current repository's test suite.

12. Pre-Demonstration Checklist

- Valid login succeeds and invalid login is rejected.
- Logout returns to /login; refreshing /login does not show Not Found.
- Protected pages redirect unauthenticated users.
- Each role can perform permitted actions, and unauthorized API requests are rejected.
- A valid work order can be created and remains after refresh.
- Invalid work-order data is rejected with a useful error.
- Job cards and material requests behave as intended.
- Inventory issuance updates stock correctly and prevents invalid stock reductions.
- AI priority scores, semantic search, and reports return sensible results.
- Audit records are created for relevant operations.
- Frontend, backend, and database connectivity are verified.
- No secrets are exposed in the repository or browser.

13. Viva Preparation

Question: What are functional requirements?

Answer: Functional requirements specify the services and operations a system must provide. In this project, examples include login, role-based access, work-order creation, job-card tracking, inventory management, audit logging, and AI-assisted reporting.

Question: What are non-functional requirements?

Answer: Non-functional requirements define quality attributes such as security, performance, reliability, usability, maintainability, compatibility, and data integrity.

Question: Where did you test the system?

Answer: We developed and debugged the application locally, tested the deployed interface on Render, inspected API requests using browser Developer Tools, and checked the PostgreSQL database through Prisma validation, migration status, and Prisma Studio.

Question: Give an example of a bug you fixed.

Answer: After logout, the deployed application displayed Not Found for /login. The static hosting rewrite was configured to serve the React entry point for application routes, and the route was retested.

Question: How did you validate the database?

Answer: We validated the Prisma schema, checked migration status, ran the seed script successfully, and inspected the resulting tables and records using Prisma Studio.

Question: Is all testing complete?

Answer: The documented checks verified selected database, deployment, routing, and validation behavior. Comprehensive role authorization, performance, load, and security coverage should only be claimed after running and recording those tests.

14. Security and Reporting Notes

- Do not publish database credentials, JWT secrets, or .env files.
- Change or disable default demo credentials before public or production use.
- Use a separate development database for testing.
- Do not run destructive Prisma commands against production.
- Record actual test outcomes; distinguish passed tests from planned tests.
- Check hosting configuration, endpoint availability, and environment variables before a live demonstration.

15. Project Summary

The project brings together a React frontend, Express backend, PostgreSQL database, Prisma ORM, role-based access controls, and AI-assisted maintenance functionality. The strongest verified evidence discussed includes successful Prisma schema validation, an up-to-date migration status, successful seeding, database inspection, and the resolution of the deployed logout route issue. Further tests should be run to establish complete coverage of permissions, performance, security, and automated behavior.